ZT Main Search: Lightweight Neighbor Rows

A byte-identical way to stop scoring the 12/13 of PSMs we discard

feat/sciex-zt-scanning

2026-07-09

1 Idea and why it is byte-identical

For ZT, meta-scan expansion makes each precursor a candidate in $2k{+}1{=}13$ bins per cycle, so deconvolution emits ≈35M PSMs/file; `collapse_to_metascans` then keeps the **one center bin** per (precursor, cycle) (≈3.2M) and, from the other 12 bins, reads **only the** `weight` **scalar**. Everything else on a non-center row — the spectral scores (`gof`, `hellinger`, residuals), the 16 fitted/shadow fragment intensities, and every downstream feature — is computed and thrown away.

The plan: compute full scores + emit a full row only for center bins; for neighbor bins emit just `(precursor_idx, scan_idx, weight)`. This is *byte-identical* because:

1. The Huber/LS *solve* that produces every `weight` is untouched (it needs all candidates).
2. The shared residual $r = \text{observed} - \sum_p w_p H_p$ is built from all weights, untouched.
3. A *center* row's scores — including its `shadow_frag*` (interference from co-eluting precursors) — read r and the neighbors' *weights*, never the neighbors' *scores*. So center rows come out identical.
4. The collapse reads identical weights (center + neighbor) and identical center-row features.

⇒ verify with an exact diff of `precursors_long.arrow`, not an A/B.

2 Current call chain

`process_scans_fused.jl` (per scan) → `compute_distance_metrics!` (→ `getDistanceMetrics`) → `score_psms!` (→ `Score!`, appends `MainSearchScoredPSM` rows) → [ZT] `collapse_to_metascans`.

(a) The scoring kernel — `spectralDistanceMetrics.jl:59`

```
59 for col in range(1, H.n)                # build shared residual r from ALL weights
60     for n in H.colptr[col]:H.colptr[col+1]-1
61         r[H.rowval[n]] += w[col]*H.nzval[n]
62     end
63 end
64 for col in 1:H.n                        # Single-pass scoring per precursor (SEPARABLE)
65     if w[col] <= zero(T)
66         spectral_scores[col] = SpectralScoresMainSearch(zero(U), ...); continue
67     end
68     ...
69     @inbounds @fastmath for i in H.colptr[col]:(H.colptr[col+1]-1)   # <-- the expensive
           body
```

```
        fitted_peak = w[col]*H.nzval[i]
        shadow_peak = fitted_peak - r[H.rowval[i]]     # reads SHARED r (all neighbors' weights)
        ... gof / hellinger / fitted_frag* / shadow_frag* ...
    end
    spectral_scores[col] = SpectralScoresMainSearch(...)
end
```

(b) The row emit — `ScoredPSMs.jl:173`

```
for i in range(1, n_vals)
    precursor_idx = UInt32(unscored_PSMs[i].precursor_idx)
    scores_idx = IDtoCOL[precursor_idx]
    if weight[scores_idx] < Float32(1e-6)
        skipped += 1; continue
    end
    ...
    scored_psms[start_idx + i - skipped] = MainSearchScoredPSM(   # 61-col row
        unscored_PSMs[i].longest_y, ..., weight[scores_idx],
        L(spectral_scores[scores_idx].gof), ...,
        H(spectral_scores[scores_idx].shadow_frag8_int), ...)
end
```

(c) The collapse gather — `metascan_collapse.jl:104`

```
for r in blk_lo:blk_hi                          # rows of one precursor, sorted by scan
    (abs(pm - Float32(cmzs[scn[r]])) <= Float32(hws[scn[r]])/2) || continue  # center test
    c = Int(scn[r])
    lo = max(blk_lo, r - L); hi = min(blk_hi, r + L)
    for rr in lo:hi
        d = Int(scn[rr]) - c
        (-k <= d <= k) && (w[d+k+1] += wt[rr])   # gather +/-k WEIGHTS from neighbor rows
    end
    push!(center_rows, r); push!(profiles, w)
end
```

3 The change, step by step

Everything below is gated on `_zt_main` (a `use_lightweight::Bool` threaded down from `library_search`); non-ZT keeps the exact current path.

Step 1 — a per-column `is_center` mask (in `process_scans_fused.jl`)

The center test is exactly `filter_to_center_bin!`'s: is this precursor's m/z in the scan's center bin. Compute it once per scan into a per-thread scratch `is_center::VectorBool` indexed by design-matrix column, using data already in scope (`scan_idx`, `prec_mzs`, `id_to_col`).

```
# after run_fused! populates id_to_col, before compute_distance_metrics!:
if use_lightweight
    cmz = getCenterMz(spectra, scan_idx); hw = getIsolationWidthMz(spectra, scan_idx)/2
    fill_is_center!(is_center, id_to_col, precs_vec, sub_indices, prec_mzs, cmz, hw)
end
# fill_is_center!: for each candidate precursor p at column col=id_to_col[p],
#   is_center[col] = !ismissing(cmz) && abs(prec_mzs[p] - cmz) <= hw
```

Step 2 — skip the scoring body for non-center columns (`getDistanceMetrics`)

Add `is_center` (optional; `nothing` ⇒ score all, i.e. non-ZT). The residual build (lines 59–65) is unchanged; only the per-precursor scoring loop is gated. Non-center `spectral_scores[col]` is never read downstream, so we simply skip it.

```
for col in 1:H.n
    if w[col] <= zero(T)
        spectral_scores[col] = SpectralScoresMainSearch(zero(U), ...); continue
    end
    if is_center !== nothing && !is_center[col]
        continue                        # <-- NEW: neighbor -> weight already in w[col];
            skip scoring
    end
    ...                                 # unchanged expensive body, only for centers (+ non
        -ZT: all)
end
```

The shadow of a center still reads the shared `r`, which was built from *all* weights including this neighbor's — hence identical.

Step 3 — route neighbors to a compact side-buffer (`Score!`)

Add `is_center` and a per-thread `NeighborWeights` SoA. Center ⇒ full `MainSearchScoredPSM` exactly as today; neighbor ⇒ 12-byte record.

```
for i in range(1, n_vals)
    precursor_idx = UInt32(unscored_PSMs[i].precursor_idx)
    scores_idx = IDtoCOL[precursor_idx]
    weight[scores_idx] < Float32(1e-6) && (skipped += 1; continue)

    if is_center !== nothing && !is_center[scores_idx]
        push!(nbr.precursor_idx, precursor_idx)      # <-- NEW compact neighbor row
        push!(nbr.scan_idx, UInt32(scan_idx))
        push!(nbr.weight, weight[scores_idx])
        skipped += 1                                 # keep scored_psms compaction
            intact
        continue
    end
    ... scored_psms[start_idx + i - skipped] = MainSearchScoredPSM(...)   # unchanged (
        centers)
end
```

```
struct NeighborWeights                  # Struct-of-Arrays: 12 B/row, Arrow-friendly
    precursor_idx::Vector{UInt32}
    scan_idx::Vector{UInt32}
    weight::Vector{Float32}
end
```

Per-thread instances are concatenated after the scan loop, exactly like `scored_psms` (`process_scans_fused.jl:135`). `library_search` returns `(psms, neighbor_weights)` for ZT.

Step 4 — collapse gathers from centers + neighbor buffer

Today's gather (2c) reads $\pm k$ weights from the one sorted PSM table. Now the $\pm k$ weights come from two sources. Build a combined, `(precursor_idx, scan_idx)`-sorted weight list — center weights from `psms.weight`, neighbor weights from the buffer — and gather against it, while the kept *rows* (and all features) remain the center table only.

```
# combined weight index (only 3 small columns, ~35M x 12 B, no 61-col rows):
cw_pid  = vcat(psms.precursor_idx, nbr.precursor_idx)
```

```
cw_scan = vcat(psms.scan_idx,      nbr.scan_idx)
cw_wt   = vcat(psms.weight,        nbr.weight)
ord = sortperm(1:length(cw_pid); by = j -> (cw_pid[j], cw_scan[j]))   # replaces Sort 2
    on wide rows
# then, per center row c (precursor pc, scan sc): walk ord within pc for scans in [sc-k,
     sc+k],
# w[scan - sc + k + 1] += cw_wt[...]   -- identical arithmetic to 2c, lighter payload.
```

The center table itself is already the reduced output; its `metascan_w_*` and 9 shape features are filled from this combined gather exactly as before.

4 What is saved

Cost today (on ≈35M rows)	After
per-precursor scoring body (2a) for all candidates	only ≈3.2M centers (∼11× less)
≈35M × 61-col `MainSearchScoredPSM` materialization	3.2M full rows + ≈32M × 12 B
`scan_competition` / `ms1` / `prepare` / `chromatogram` features on 35M	on 3.2M
Sort 1 (`permute_by_precursor`, 4.2 s, 61-col)	gone; one sort on the 12 B weight list

The Huber/LS solve and the residual build (legitimately all-candidate) are unchanged.

5 Verification and rollout

- **Byte-identical check:** run the 3 ZT files with the flag off vs on; require `precursors_long.arrow` to be *identical* (same precursor_idx set, same `peak_area`, `gof`, `lgbm_prob`, ...). Any diff is a bug in the center predicate or the gather, not an expected shift.
- **Guard:** the center predicate must match `filter_to_center_bin!` and the collapse's center test *exactly* (same `cmz`/`hw`), or a row could be "center" in the emit but "neighbor" in the collapse (lost) or vice versa (duplicated). One shared helper `is_center_bin(prec_mz, cmz, hw)` used in all three.
- **Rollout:** env/ZT-gated (`use_lightweight = _zt_main`); non-ZT takes the `is_center === nothing` path and is untouched. Land Step 2 first (skip scoring, still emit all rows) to confirm identical scores, then Steps 3–4 (compact neighbors).
- **Constraint (from the shadow discussion):** this forecloses computing any *new* multi-precursor feature after the collapse — neighbor rows no longer carry scores. Any such feature must be produced inside `getDistanceMetrics` (at solve time) before neighbors are dropped.